

Git --ours and --theirs Explained

Understanding 'Ours' and 'Theirs'

When Git encounters a merge conflict, it keeps multiple versions of the conflicting code. The terms **--ours** and **--theirs** let you choose which version to keep.

During a Merge

Suppose you are currently on the **main** branch and run:

git merge feature-login

Ours = the branch you currently have checked out (main)

Theirs = the branch being merged into yours (feature-login)

Conflict Example

```
<<<<<<< HEAD
Hello from Main
=====
Hello from Feature
>>>>>>> feature-login
```

Keep Ours

```
git checkout --ours file.txt
git add file.txt
git commit
```

Result: Keeps the current branch version.

Keep Theirs

```
git checkout --theirs file.txt
git add file.txt
git commit
```

Result: Keeps the incoming branch version.

Apply to All Conflicted Files

```
Keep ours:
git checkout --ours .
git add .
git commit

Keep theirs:
git checkout --theirs .
git add .
git commit
```

Important Difference During Rebase

During a merge:

Ours = Current branch

Theirs = Incoming branch

During a rebase:

Ours = Branch you're rebasing onto

Theirs = Your rebased commit

Classroom Example

Student A changes a file and pushes it.

Student B changes the same file locally.

When Student B performs git pull, a conflict occurs.

Keep Student B's local version:

`git checkout --ours training.txt`

Keep Student A's remote version:

`git checkout --theirs training.txt`

Memory Trick

Merge Scenario:

Ours = My current branch

Theirs = The other branch

Think: 'I'm standing on my branch right now, so that's OURS. The branch coming toward me is THEIRS.'